

The Pudding

Easier scrollytelling with position sticky

Leaning on CSS to simplify the process.

By [Elaina Natario](#) and [Russell Samora](#)

June 2018

We've written a lot about scrollytelling here at The Pudding. We've covered everything from a [library comparison](#), to [responsive best practices](#), to a [deep-dive into Scrollama](#). But there is always room for improvement. One of the biggest implementation pains with scrollytelling is the *sticky graphic pattern*, whereby the graphic scrolls into view, becomes “stuck” for a duration of steps, then exits and “unsticks” when the steps conclude.

This post will focus on the easiest solution we've come up with yet: offloading the sticky complexity to CSS, using `position: sticky`.

Why use Sticky?

The short version: when using the sticky graphic pattern, you need a bunch of JavaScript to handle the stuck state, dimensions, etc.. With this approach, that is all done with (minimal) CSS. This means less bugs, less maintenance, and more happiness.

What is Sticky?

Sticky positioning is the unapologetic love child of `position: relative` and `position: fixed` (in which said love child grows up to do bigger and better things while still retaining the lessons of its parents). An element with a `position: sticky` declaration remains static in the document until a certain threshold is reached, and then it becomes fixed once scrolled to that threshold. A *threshold* is defined by any directional declaration such as: `top: 0;`, which becomes fixed once the element reaches the top edge of its parent.

A sticky element is always relatively positioned to its parent (much like `position: absolute;`). This means that these elements will stick and unstick only within the bounds of its parent element, not the viewport (making our job easier); it also means that the thresholds are marked by the edges of the parent. The great thing about this constraint is that you can control the overlap (or lack thereof) of multiple stuck elements.

In Action

And now for the good stuff. The first example here uses the classic side-by-side approach. Scroll on.

Bar is 10%

Bar is 90%

Bar is 50%

The second example uses the text overlay approach. Keep on scrolling!

Bar is 10%

Bar is 90%

Bar is 50%

The sticky graphic is entirely handled by CSS, while the only thing done in JavaScript is handling the step triggers. We chose to use [enter-view.js](#) here because it has an incredibly simple interface, super lightweight, and Russell wrote it 😊. However, any trigger library will work, be it [Waypoints](#), [Scrollama](#), or another library.

The Code

Below is the core HTML, CSS and JavaScript that makes this work, excluding the code for aesthetics. You can view the full code for these examples on [codepen](#), or [github](#) for the the full article.

```
<!-- step text -->
<article>
  <div class='step' data-index='0'>
    <p>Step text</p>
  </div>
  <!-- ...more steps -->
</article>

</div>
```

```
article {
  position: relative;
}
figure.sticky {
  position: -webkit-sticky;
  position: sticky;
  top: 0;
}
article {
  position: relative;
}
figure.sticky {
  position: -webkit-sticky;
  position: sticky;
  top: 0;
}
```



```
enterView({
  selector: stepSel.nodes(),
  offset: 0.5,
  enter: el => {
    const index = +d3.select(el).attr('data-index');
    updateChart(index);
  },
  exit: el => {
    let index = +d3.select(el).attr('data-index');
    index = Math.max(0, index - 1);
    updateChart(index);
  }
});
enterView({
  selector: stepSel.nodes(),
  offset: 0.5,
  enter: el => {
    const index = +d3.select(el).attr('data-index');
    updateChart(index);
  },
  exit: el => {
    let index = +d3.select(el).attr('data-index');
    index = Math.max(0, index - 1);
    updateChart(index);
  }
});
```

Wrap Up

You can still implement this behavior in JavaScript, but `position: sticky` has a few core benefits which give it the upper hand when it comes to development. Scroll listeners in JavaScript aren't always the best solution: they can inhibit performance and are sometimes quite janky. Depending on the speed of scroll and the device someone is using, the listener might fall out-of-sync and need to catch up with a scroll, causing an element to jump into place instead of displaying a smooth locking behavior. Beyond performance concerns, it's much simpler to write two lines of CSS than a bunch of JavaScript.

But is it well-supported? Yes it is! Apart from a few minor bugs with `thead` and `tr` elements, it's good-to-go across the latest versions of all major browsers. IE doesn't

support this feature (*ugh, of course...*), but there are a few polyfills and companions out there with loose support for certain directional thresholds. If polyfills aren't your jam or the ones available just aren't cutting it, you can always... do nothing! The beauty of `position: sticky` is that it has built-in graceful degradation; if a browser doesn't support it, the element will stay static in the source order it was added. And if you want to get extra fancy, you can use a feature query `@supports (position: sticky)` and implement a fallback layout.

PICKING UP WHAT WE'RE PUDDING DOWN?

Common Threads

In pursuit of democracy

What City is the Microbrew Capital
of the US?

Where International Communities
Cluster

Get the latest emailed to you.

JOIN

Support us on Patreon.

HELP FUND US

The Pudding is a digital publication that explains

ABOUT



ideas debated in culture with visual essays.

Reach out: sup@pudding.cool or [@puddingviz](https://twitter.com/puddingviz).

**THE PUDDING® IS MADE IN BROOKLYN, NY; SEATTLE, WA; AND GREAT
BARRINGTON, MA.**